Specialized data structures in Go

The data structures you reach for when the basics aren't quite right: sets, multimaps, LRU caches, bloom filters, skip lists, and union-find. Each has a specific use case and tradeoff.

Sets

Go has no built-in set type. The idiomatic approach is `map[T]struct{}` because `struct{}` takes zero bytes.

Basic set with map

```
set := map[string]struct{}{}
set["a"] = struct{}{}
set["b"] = struct{}{}

if _, has := set["a"]; has {
    fmt.Println("present")
}

delete(set, "a")
len(set)                          // count
```

`struct{}` is the canonical empty value. `bool` works too but uses 1 byte per element.

Generic set type

```
type Set[T comparable] map[T]struct{}

func NewSet[T comparable](items ...T) Set[T] {
    s := Set[T]{}
    for _, v := range items { s[v] = struct{}{} }
    return s
}

func (s Set[T]) Add(v T)          { s[v] = struct{}{} }
func (s Set[T]) Delete(v T)       { delete(s, v) }
func (s Set[T]) Has(v T) bool     { _, ok := s[v]; return ok }
func (s Set[T]) Len() int         { return len(s) }

func (s Set[T]) ToSlice() []T {
    out := make([]T, 0, len(s))
    for k := range s { out = append(out, k) }
    return out
}

// set ops
func (s Set[T]) Union(other Set[T]) Set[T] {
    out := Set[T]{}
    for k := range s     { out[k] = struct{}{} }
    for k := range other { out[k] = struct{}{} }
    return out
}

func (s Set[T]) Intersect(other Set[T]) Set[T] {
    out := Set[T]{}
    if len(other) < len(s) { s, other = other, s }   // iterate smaller
    for k := range s {
        if _, ok := other[k]; ok { out[k] = struct{}{} }
    }
    return out
}

func (s Set[T]) Difference(other Set[T]) Set[T] {
    out := Set[T]{}
    for k := range s {
        if _, ok := other[k]; !ok { out[k] = struct{}{} }
    }
    return out
}

func (s Set[T]) IsSubset(other Set[T]) bool {
    for k := range s {
        if _, ok := other[k]; !ok { return false }
    }
    return true
}
```

For shared code, `github.com/deckarep/golang-set/v2` gives you a popular, well-tested generic set.

Sorted set

For ordered iteration, pair a `[]T` with a `map[T]int` (index lookup):

```
type SortedSet[T cmp.Ordered] struct {
    items []T
    idx   map[T]int
}

func (s *SortedSet[T]) Add(v T) {
    if _, ok := s.idx[v]; ok { return }
    i, _ := slices.BinarySearch(s.items, v)
    s.items = slices.Insert(s.items, i, v)
    for j := i; j < len(s.items); j++ { s.idx[s.items[j]] = j }
}
```

Insertion is O(n) due to slice shift. For a true sorted set with O(log n) operations, use `google/btree` or a skip list.

Bitset (set of small integers)

For sets of nonnegative integers with a known upper bound, a bitset uses 1 bit per element. Much smaller and faster than a map.

```
type BitSet []uint64

func NewBitSet(n int) BitSet {
    return make(BitSet, (n+63)/64)
}

func (b BitSet) Set(i int)     { b[i/64] |= 1 << uint(i%64) }
func (b BitSet) Clear(i int)   { b[i/64] &^= 1 << uint(i%64) }
func (b BitSet) Has(i int) bool { return b[i/64]&(1<<uint(i%64)) != 0 }

func (b BitSet) Count() int {
    n := 0
    for _, w := range b { n += bits.OnesCount64(w) }
    return n
}
```

Union and intersection are word-level operations: `out[i] = a[i] | b[i]`. Extremely fast.

Use bitsets for: visited flags in graph algorithms over dense node IDs, feature flags, bitmap indexes, set operations on millions of small IDs.

Multimap

Map where each key can have multiple values. Go doesn't have one; use `map[K][]V`.

```
m := map[string][]int{}
m["alice"] = append(m["alice"], 1)
m["alice"] = append(m["alice"], 2)
m["bob"] = append(m["bob"], 3)

for _, v := range m["alice"] { ... }
```

Generic helper:

```
type Multimap[K comparable, V any] map[K][]V

func (m Multimap[K, V]) Add(k K, v V)        { m[k] = append(m[k], v) }
func (m Multimap[K, V]) Get(k K) []V         { return m[k] }
func (m Multimap[K, V]) Remove(k K)          { delete(m, k) }
func (m Multimap[K, V]) Has(k K) bool        { _, ok := m[k]; return ok }
```

Use cases: - Index by attribute (group items by category). - One-to-many relationships. - Inverted index (term -> list of doc IDs).

LRU cache

Least Recently Used cache with bounded size. The classic implementation: doubly linked list (recency order) + map (O(1) lookup).

```go
type LRU[K comparable, V any] struct {
    cap   int
    items map[K]*list.Element
    order *list.List           // front = most recent
}

type entry[K comparable, V any] struct {
    key K
    val V
}

func NewLRU[K comparable, V any](cap int) *LRU[K, V] {
    return &LRU[K, V]{
        cap:   cap,
        items: map[K]*list.Element{},
        order: list.New(),
    }
}

func (c *LRU[K, V]) Get(k K) (V, bool) {
    var zero V
    e, ok := c.items[k]
    if !ok { return zero, false }
    c.order.MoveToFront(e)
    return e.Value.(*entry[K, V]).val, true
}

func (c *LRU[K, V]) Add(k K, v V) {
    if e, ok := c.items[k]; ok {
        e.Value.(*entry[K, V]).val = v
        c.order.MoveToFront(e)
        return
    }
    if c.order.Len() == c.cap {
        oldest := c.order.Back()
        if oldest != nil {
            c.order.Remove(oldest)
            delete(c.items, oldest.Value.(*entry[K, V]).key)
        }
    }
    e := c.order.PushFront(&entry[K, V]{key: k, val: v})
    c.items[k] = e
}

func (c *LRU[K, V]) Len() int { return c.order.Len() }
```

Properties

- `Get` : O(1).
- `Add` : O(1).
- Capacity bounds memory.

- Eviction: least recently used (oldest in the list).

Variants

- **TTL** (time to live): expire entries after a duration. Pair LRU with a periodic sweeper or check on access.
- **LFU** (least frequently used): evict the least-accessed. More complex; needs a frequency counter and a heap or buckets.
- **ARC** (Adaptive Replacement Cache): adapts between LRU and LFU. Used by ZFS, PostgreSQL.
- **2Q**: two queues for “seen once” and “seen twice.” Better hit ratio than plain LRU for some workloads.

For most caching, plain LRU with TTL is enough.

Libraries

Library	Notes
`github.com/hashicorp/golang-lru/v2`	Generic, well-tuned, TTL variant
`github.com/dgraph-io/ristretto`	High-throughput, async writes
`github.com/coocood/freecache`	Off-heap, predictable GC
`github.com/VictoriaMetrics/fastcache`	Fast, big workloads

Hashicorp’s library is the easy default. Ristretto if you need to push throughput.

Bloom filter

A probabilistic set. Tells you “definitely not in the set” or “maybe in the set.” Very memory efficient.

```go
type BloomFilter struct {
    bits []uint64
    k    int                // number of hash functions
    m    int                // number of bits
}

func NewBloomFilter(expected int, fpRate float64) *BloomFilter {
    m := int(-float64(expected) * math.Log(fpRate) / math.Pow(math.Log(2), 2))
    k := int(float64(m) / float64(expected) * math.Log(2))
    return &BloomFilter{
        bits: make([]uint64, (m+63)/64),
        k:    k,
        m:    m,
    }
}

func (b *BloomFilter) hashes(data []byte) []uint32 {
    h1, h2 := hash128(data)
    out := make([]uint32, b.k)
    for i := 0; i < b.k; i++ {
        out[i] = uint32(h1 + uint64(i)*h2) % uint32(b.m)
    }
    return out
}

func (b *BloomFilter) Add(data []byte) {
    for _, pos := range b.hashes(data) {
        b.bits[pos/64] |= 1 << (pos % 64)
    }
}

func (b *BloomFilter) MayContain(data []byte) bool {
    for _, pos := range b.hashes(data) {
        if b.bits[pos/64]&(1<<(pos%64)) == 0 {
            return false
        }
    }
    return true
}
```

Properties

- False negatives: never (0% chance).
- False positives: tunable (typically 1-5%).
- Memory: about 1.44 * log2(1/p) bits per item for false positive rate p.
- Adds and queries: O(k).
- No removal in standard bloom filter; use counting bloom filter for that.

When to use

- Pre-filter expensive lookups (cache before disk, before network).

- Spam filters.
- Detecting duplicate URLs in a crawler.
- Deciding whether to check disk in an LSM-tree database (RocksDB, Cassandra).
- HyperLogLog's cousin for set cardinality estimation.

Libraries

`github.com/bits-and-blooms/bloom/v3` is a solid implementation.

```
import "github.com/bits-and-blooms/bloom/v3"

filter := bloom.NewWithEstimates(1000000, 0.01)   // 1M items, 1% FPR
filter.Add([]byte("hello"))
filter.Test([]byte("hello"))                       // probably true
```

Skip list

A probabilistic alternative to balanced trees. Each node has multiple "express lanes" at random heights. Search, insert, and delete are O(log n) expected.

```
type skipNode[T cmp.Ordered] struct {
    value T
    next  []*skipNode[T]       // length = level
}

type SkipList[T cmp.Ordered] struct {
    head    *skipNode[T]
    level   int
    maxLevel int
    p       float64            // promotion probability
    size    int
    rng     *rand.Rand
}
```

Insert: pick a random level; insert at every level up to that. Search: start at top level, move right until next > target, drop down a level.

Why skip lists exist

- Simpler to implement than red-black or AVL trees (no rotations).
- Good for concurrent variants (lock-free skip lists are easier than lock-free balanced trees).
- Used in Redis (sorted sets), LevelDB, and many memtable implementations.

Go libraries

- `github.com/MauriceGit/skiplist` - basic skiplist.
- For ordered keyed structures, `google/btree` is usually a faster Go choice in single-threaded workloads.

Skip lists shine in concurrent scenarios (lock-free reads while writers update). For single-threaded ordered structures, btree wins on cache performance.

Union-find (disjoint set)

Tracks a partition of elements into disjoint sets. Two operations: - `Find(x)` : which set does x belong to? - `Union(x, y)` : merge the sets containing x and y.

With path compression and union by rank, both operations run in nearly O(1) amortized (technically O(α(n)) where α is the inverse Ackermann function).

```go
type UnionFind struct {
    parent []int
    rank   []int
    size   []int
}

func NewUnionFind(n int) *UnionFind {
    uf := &UnionFind{
        parent: make([]int, n),
        rank:   make([]int, n),
        size:   make([]int, n),
    }
    for i := range uf.parent {
        uf.parent[i] = i
        uf.size[i]   = 1
    }
    return uf
}

func (uf *UnionFind) Find(x int) int {
    for uf.parent[x] != x {
        uf.parent[x] = uf.parent[uf.parent[x]]   // path compression
        x = uf.parent[x]
    }
    return x
}

func (uf *UnionFind) Union(x, y int) bool {
    rx, ry := uf.Find(x), uf.Find(y)
    if rx == ry { return false }
    if uf.rank[rx] < uf.rank[ry] { rx, ry = ry, rx }
    uf.parent[ry] = rx
    uf.size[rx]  += uf.size[ry]
    if uf.rank[rx] == uf.rank[ry] { uf.rank[rx]++ }
    return true
}

func (uf *UnionFind) Connected(x, y int) bool {
    return uf.Find(x) == uf.Find(y)
}

func (uf *UnionFind) Size(x int) int {
    return uf.size[uf.Find(x)]
}
```

Use cases

- Kruskal's MST.
- Connected components in dynamic graphs (edges added over time).
- Cycle detection.
- Network connectivity.

- Image segmentation.
- Percolation problems.
- “Friend of friend” relationships in social networks.

Variants

- **Union by size** instead of rank: nearly identical performance.
- **Weighted union-find**: each element knows its size; useful for finding the largest component.
- **Persistent union-find**: keeps history of union operations.

Concurrent versions

Concurrent map: sync.Map

The stdlib’s `sync.Map` is optimized for two patterns: - Each key written once and read many times. - Different goroutines work on disjoint key sets.

For balanced read/write, a regular map + `sync.RWMutex` is usually faster. Benchmark in your access pattern.

```
var m sync.Map
m.Store("k", v)
v, ok := m.Load("k")
m.Delete("k")
m.Range(func(k, v any) bool { return true })
```

Concurrent LRU

Wrap the LRU with a mutex. For high contention, shard by key hash into N LRUs, each with its own lock.

```
type ShardedLRU[K comparable, V any] struct {
    shards [256]*lockedLRU[K, V]
}

func (s *ShardedLRU[K, V]) shard(k K) *lockedLRU[K, V] {
    h := hash(k)
    return s.shards[h%256]
}
```

`fastcache` and `ristretto` use sharding internally for this reason.

Concurrent skip list / lock-free structures

Lock-free queues, stacks, hashmaps exist but are tricky to get right and rarely faster than well-sharded locked versions in real Go workloads. Reach for these only when profiling demands it.

Probabilistic data structures (other useful ones)

Structure	Tells you	Memory	Use
Bloom filter	Probably has / definitely not	very low	Pre-filter
Counting Bloom filter	Same, but supports remove	low	Mutable membership
Cuckoo filter	Same as bloom, supports delete	low	Bloom alternative
HyperLogLog	Cardinality estimate	~12 KB for 0.81% error on billions	Distinct counts
Count-Min Sketch	Frequency estimate	low, configurable	Top-K, heavy hitters
MinHash	Set similarity	low	Document dedup, recommendations
t-digest	Quantile estimate	low	p99 latency without storing every sample

Most have well-maintained Go libraries. `axiomhq/hyperloglog`, `tidwall/hyperloglog`, `dgryski/go-bloomf`, etc.

Choosing among them

Need	Pick
Distinct membership, cheap	Map[K]struct{}
Distinct membership, huge data, OK with false positives	Bloom filter
Member count	`len(map)`
Approximate distinct count, low memory	HyperLogLog
Order matters	Sorted slice, btree, or skip list
Frequency top-K	Count-Min Sketch + heap
Disjoint set tracking	Union-find
Bounded LRU cache	hashicorp/golang-lru
Cache with high-throughput writes	Ristretto
Set operations on dense IDs	Bitset
One-to-many	`map[K][]V` or generic Multimap
Concurrent ordered structure	Skip list or sharded btree
Concurrent map	RWMutex + map, or sharded, or sync.Map
In-memory time series with quantiles	t-digest

Performance notes

Set lookup costs

- `map[K]struct{}` lookup: ~10-30 ns.
- `[]K` linear search: ~ns per element; faster than map for n < 16 or so.
- `sort.Search` on sorted slice: ~10-30 ns for moderate n.
- Bitset bit check: ~1 ns.

For small known-bounded sets, a sorted slice with binary search often beats a map.

LRU costs

- `Get` from a well-tuned LRU: 50-100 ns.

- Eviction: similar.
- The hot path is the lock + map lookup + list move. Each takes nanoseconds.

`ristretto` claims 10-100x throughput over locked LRU for write-heavy workloads via batched async updates.

Bloom filter costs

- `Add`: ~50-200 ns per call (k hashes).
- `Test`: ~50-200 ns per call.

The cost scales with k (number of hash functions), typically 7-10.

Best practices

1. **Use `map[K]struct{}` for sets.** The zero-byte value is the Go idiom.
2. **Generic Set[T] is fine.** Don't reach for libraries unless you need set algebra heavily.
3. **Bitset for dense integer IDs.** 64x smaller than a `[]bool`.
4. **LRU library, not hand-rolled, for production.** `hashicorp/golang-lru` is solid.
5. **Bloom filter when "probably yes / definitely no" is good enough.** Saves an expensive lookup.
6. **Union-find for connectivity over edges added incrementally.**
7. **Sharded structures for high concurrency.** A single mutex on a hot path is a bottleneck.
8. **Pick the data structure to match the access pattern.** "I need a set, what's the fastest set?" is the wrong question. "I have these queries, what serves them?" is the right one.
9. **Benchmark, don't speculate.** Cache locality often wins over textbook asymptotics for n < ~10000.
10. **Test edge cases:** empty container, single element, full capacity, just-evicted entries.

Common gotchas

Gotcha	Symptom	Fix
`map[K]bool` for set	1 byte per entry instead of 0	Use `map[K]struct{}`
Forgot to size LRU	OOM	Set capacity at construction
LRU `Get` doesn't update order	Eviction picks wrong key	`MoveToFront` on access
Bloom filter "has" check is "definitely"	False positive treated as truth	Pair with authoritative lookup
Bloom filter for "definitely not in" cache	False positives = unnecessary checks	Tune size/k for FPR
Union-find without path compression	O(n) worst case	Compress in `Find`
Sets with concurrent writes	Race	Mutex or sync.Map
Skip list without random level	Degenerates to linked list	Use a real PRNG
Counting bloom filter with overflow	Wrong counts	Saturating counters
LFU with O(n) update	Slow under load	Use buckets or heap
Multimap with empty list left after delete	Memory leak (key remains)	Delete key when slice empty
Bitset out of range	Panic	Bounds check or grow
Concurrent LRU with stale snapshot	Wrong eviction	Lock the whole get/update path
sync.Map for balanced workload	Slower than RWMutex+map	Use the right tool
HyperLogLog merge across versions	Subtle bias	Use one library version
Cuckoo filter under heavy load	Insertion can fail	Plan resize strategy

Quick reference

Structure	Go implementation
Set (general)	`map[K]struct{}`
Generic set	`type Set[T comparable] map[T]struct{}`
Sorted set	`google/btree` or sorted slice
Bitset	`[]uint64` with bit indexing
Multimap	`map[K][]V`
LRU cache	`hashicorp/golang-lru/v2`
LRU + TTL	`golang-lru/v2/expirable`
Cache (high throughput)	`dgraph-io/ristretto`
Off-heap cache	`coocood/freecache`
Bloom filter	`bits-and-blooms/bloom/v3`
Counting bloom	`bits-and-blooms/bloom/v3` (counting variant)
Cuckoo filter	`linvon/cuckoo-filter`
HyperLogLog	`axiomhq/hyperloglog`
Count-Min Sketch	`dgryski/go-topk`, custom
t-digest	`influxdata/tdigest`
Skip list	`MauriceGit/skiplist`
Union-find	Hand-rolled (small code)
Concurrent map	`sync.Map` (specific patterns) or RWMutex + map
Sharded structure	Pick N shards, hash key to shard
Probabilistic distinct count	HyperLogLog
Probabilistic membership	Bloom or Cuckoo filter
Heavy hitter detection	Count-Min Sketch + heap
Latency quantile estimate	t-digest